

Patrones de Diseño Arquitectónico

Aprendiz:

Jair Alfonso Arias Cueca

Instructor:

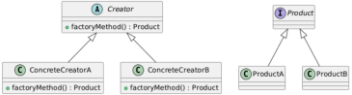
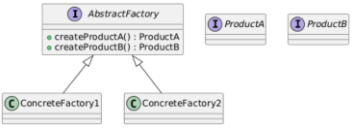
Ing. Néstor Montaña

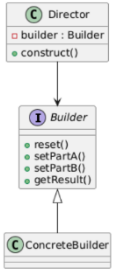
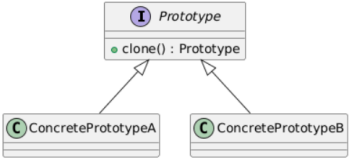
Tecnólogo Análisis y Desarrollo de Software

Ficha: 3064241

SENA Centro de Diseño y Metrología

Patrones Creacionales

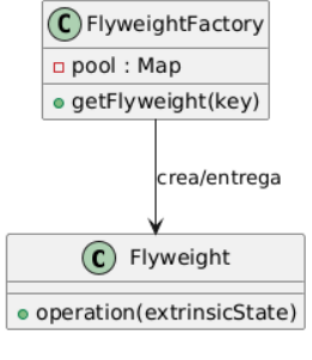
Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Singleton	Garantiza una única instancia global y un punto de acceso centralizado.	Creacional		<pre>javascript\nclass Singleton {\n constructor() { if\n (Singleton.instance) return\n Singleton.instance;\n Singleton.instance = this;\n }\n}</pre>
Factory Method	Permite crear objetos sin especificar la clase exacta.	Creacional		<pre>javascript\nclass Creator {\n factoryMethod(t) {return\n t==='A'?new A():new\n B();}\n}</pre>
Abstract Factory	Crea familias de objetos relacionados sin especificar sus clases concretas.	Creacional		<pre>javascript\nclass UIFactory {\n createButton() {\n createMenu() {} }\n}</pre>

Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Builder	Separa la construcción compleja de un objeto de su representación final.	Creacional	 <pre> classDiagram class Director { Builder builder construct() } class Builder { <<abstract>> reset() setPartA() setPartB() getResult() } class ConcreteBuilder Director --> Builder : builder Builder < -- ConcreteBuilder </pre>	<pre> javascript\nclass Builder{ setA(){} setB(){} build(){} } </pre>
Prototype	Crea objetos clonando instancias existentes.	Creacional	 <pre> classDiagram class Prototype { <<abstract>> clone() : Prototype } class ConcretePrototypeA class ConcretePrototypeB Prototype < -- ConcretePrototypeA Prototype < -- ConcretePrototypeB </pre>	<pre> javascript\ncconst proto={a:1}; const obj=Object.create(proto);\n </pre>

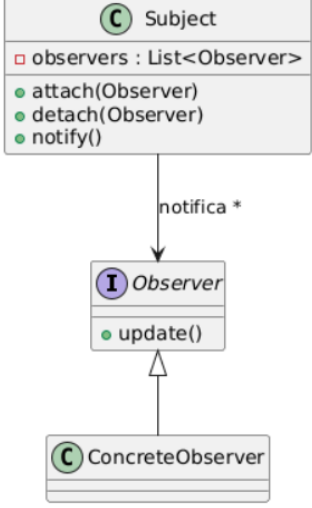
Patrones Estructurales

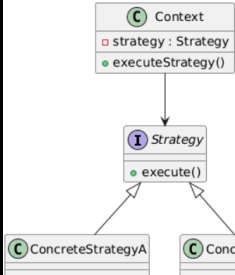
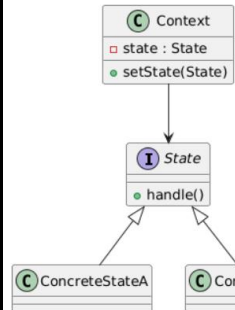
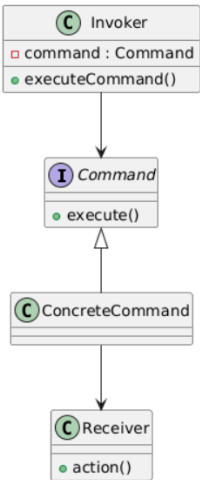
Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Adapter	Permite compatibilidad entre interfaces diferentes.	Estructural	<pre> classDiagram class Target { +request() } class Adapter { +adaptee : Adaptee +request() } class Adaptee { +specificRequest() } Target < -- Adapter Adapter --> Adaptee </pre>	<pre> javascript\n class Adapter { request(){return new Old().specific();} } </pre>
Decorator	Agrega responsabilidades a un objeto dinámicamente.	Estructural	<pre> classDiagram class Component { +operation() } class ConcreteComponent { } class Decorator { +component : Component +operation() } class ConcreteDecoratorA { } class ConcreteDecoratorB { } Component < -- ConcreteComponent Component < -- Decorator Decorator < -- ConcreteDecoratorA Decorator < -- ConcreteDecoratorB Decorator --> Component </pre>	<pre> javascript\n function deco(fn){return ()=> fn()+' decorado';}\n </pre>
Facade	Proporciona una interfaz simple para un sistema complejo.	Estructural	<pre> classDiagram class Facade { +operation() } class SubsystemA { } class SubsystemB { } Facade --> SubsystemA Facade --> SubsystemB </pre>	<pre> javascript\n class Facade{start(){new A().a();new B().b();}}\n </pre>

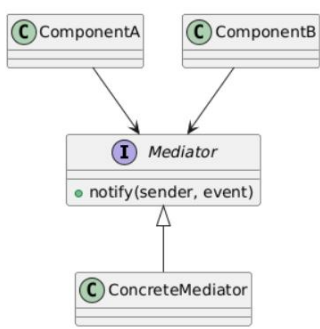
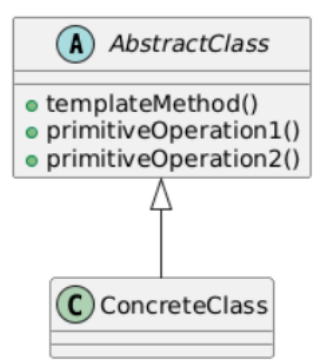
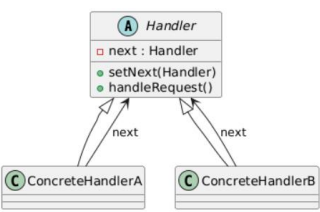
Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Proxy	Controla el acceso a un objeto real agregando un intermediario.	Estructural	<pre> classDiagram class Subject { <<interface>> request() } class Proxy { request() realSubject : RealSubject } class RealSubject { request() } Subject < -- Proxy Subject < -- RealSubject Proxy --> RealSubject : request() </pre>	<pre> javascript\nconst proxy=new Proxy(obj,{get(t,p){return t[p];}});\n </pre>
Composite	Permite tratar objetos individuales y compuestos de forma uniforme.	Estructural	<pre> classDiagram class Component { <<interface>> operation() } class Leaf { operation() } class Composite { operation() children : List<Component> add(Component) remove(Component) } Component < -- Leaf Component < -- Composite Composite --> Component : contiene * </pre>	<pre> javascript\nclass Component{add() {} operation(){} }\n </pre>
Bridge	Separa una abstracción de su implementación.	Estructural	<pre> classDiagram class Abstraction { <<interface>> operation() implementor : Implementor } class Implementor { operationImpl() } class ConcreteAbstraction { operation() } class ConcreteImplementorA { operationImpl() } class ConcreteImplementorB { operationImpl() } Abstraction < -- ConcreteAbstraction Implementor < -- ConcreteImplementorA Implementor < -- ConcreteImplementorB Abstraction --> Implementor : operation() </pre>	<pre> javascript\nclass Abstraction{constructor(i mp){this.imp=imp;} run(){this.imp.run();}}\n </pre>

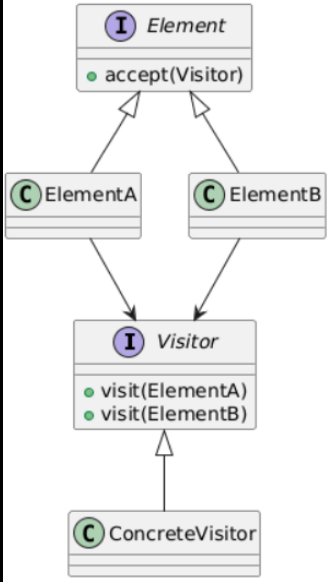
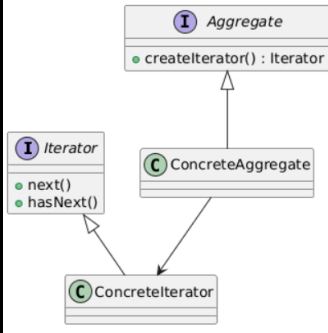
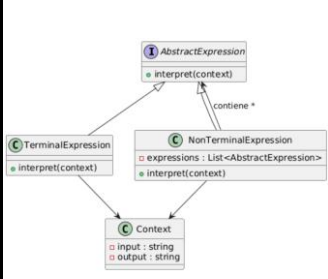
Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Flyweight	Optimiza memoria compartiendo objetos repetidos.	Estructural	 <pre> classDiagram class FlyweightFactory { pool : Map getFlyweight(key) } class Flyweight { operation(extrinsicState) } FlyweightFactory --> Flyweight : crea/entrega </pre>	<pre> javascript\nclass Fly{ constructor(s){this.s=s;} }\n </pre>

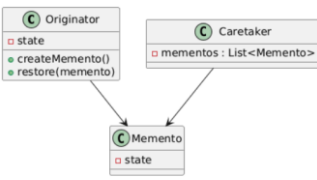
Patrones de Comportamiento

Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Observer	Notifica automáticamente a observadores sobre cambios de estado.	Comportamiento	 <pre> classDiagram class Subject { observers : List<Observer> attach(Observer) detach(Observer) notify() } class Observer { update() } class ConcreteObserver { } Subject --> Observer : notifica * Observer < -- ConcreteObserver </pre>	<pre> javascript\nclass Sub{obs=[];notify(d){this .obs.forEach(o=>o.up(d)); }}\n </pre>

Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Strategy	Permite cambiar algoritmos dinámicamente.	Comportamiento	 <pre> classDiagram class Context { Strategy strategy executeStrategy() } class Strategy { execute() } class ConcreteStrategyA class ConcreteStrategyB Context --> Strategy Strategy < -- ConcreteStrategyA Strategy < -- ConcreteStrategyB </pre>	<pre> javascript\nclass Ctx {set(s){this.s=s;} run(){return this.s.exec();}}\n </pre>
State	Permite cambiar el comportamiento de un objeto según su estado interno.	Comportamiento	 <pre> classDiagram class Context { State state setState(State) } class State { handle() } class ConcreteStateA class ConcreteStateB Context --> State State < -- ConcreteStateA State < -- ConcreteStateB </pre>	<pre> javascript\nclass StateA {handle(){return 'A';}}\n </pre>
Command	Convierte acciones en objetos independientes que pueden deshacerse o almacenarse.	Comportamiento	 <pre> classDiagram class Invoker { Command command executeCommand() } class Command { execute() } class ConcreteCommand class Receiver { action() } Invoker --> Command Command < -- ConcreteCommand ConcreteCommand --> Receiver </pre>	<pre> javascript\nclass Cmd {exec(){} }\n </pre>

Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Mediator	Reduce dependencias permitiendo que objetos se comuniquen mediante un mediador.	Comportamiento	 <pre> classDiagram class ComponentA class ComponentB class Mediator { <<interface>> +notify(sender, event) } class ConcreteMediator ComponentA --> Mediator ComponentB --> Mediator ConcreteMediator -- > Mediator </pre>	<pre> javascript\nclass Mediator{send(msg,to){to .receive(msg);}}\n </pre>
Template Method	Define el esqueleto de un algoritmo y permite que subclasses implementen detalles.	Comportamiento	 <pre> classDiagram class AbstractClass { <<abstract>> +templateMethod() +primitiveOperation1() +primitiveOperation2() } class ConcreteClass ConcreteClass -- > AbstractClass </pre>	<pre> javascript\nclass T{run(){this.a();this.b();} }\n </pre>
Chain of Responsibility	Envía solicitudes a través de una cadena de objetos hasta que uno la maneja.	Comportamiento	 <pre> classDiagram class Handler { <<abstract>> +next : Handler +setNext(Handler) +handleRequest() } class ConcreteHandlerA class ConcreteHandlerB ConcreteHandlerA -- > Handler ConcreteHandlerB -- > Handler ConcreteHandlerA --> ConcreteHandlerB : next ConcreteHandlerB --> ConcreteHandlerA : next </pre>	<pre> javascript\nclass H{setNext(n){this.n=n;} handle(r){if(this.n)this.n.h andle(r);}}\n </pre>

Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Visitor	Añade operaciones a objetos sin modificar sus clases.	Comportamiento	 <pre> classDiagram class Element { <<interface>> +accept(Visitor) } class ElementA class ElementB class Visitor { <<interface>> +visit(ElementA) +visit(ElementB) } class ConcreteVisitor Element < -- ElementA Element < -- ElementB Visitor < -- ConcreteVisitor ElementA --> Visitor ElementB --> Visitor </pre>	<pre> javascript\nclass Visitor{visit(obj){obj.accept(this);}}\n </pre>
Iterator	Proporciona un método para recorrer colecciones secuencialmente.	Comportamiento	 <pre> classDiagram class Aggregate { <<interface>> +createIterator() Iterator } class Iterator { <<interface>> +next() +hasNext() } class ConcreteAggregate class ConcreteIterator Aggregate < -- ConcreteAggregate Iterator < -- ConcreteIterator ConcreteAggregate --> ConcreteIterator </pre>	<pre> javascript\nfor(const i of [1,2,3]) console.log(i);\n </pre>
Interpreter	Define un lenguaje ejecutable mediante reglas gramaticales.	Comportamiento	 <pre> classDiagram class AbstractExpression { <<interface>> +interpret(context) } class TerminalExpression { +interpret(context) } class NonTerminalExpression { +expressions : List<AbstractExpression> +interpret(context) } class Context { +input : string +output : string } AbstractExpression < -- TerminalExpression AbstractExpression < -- NonTerminalExpression NonTerminalExpression "contiene *" --> AbstractExpression Context --> TerminalExpression Context --> NonTerminalExpression </pre>	<pre> javascript\nclass Expr{interp(){} }\n </pre>

Nombre del Patrón	Definición	Clasificación	Ejemplo en UML	Implementación en JavaScript
Memento	Permite guardar y restaurar estados anteriores de un objeto.	Comportamiento	 <pre> classDiagram class Originator { state createMemento() restore(memento) } class Caretaker { mementos : List<Memento> } class Memento { state } Originator --> Memento Caretaker --> Memento </pre>	<pre> javascript\nclass Origin {save() {return {...this}}} \n </pre>